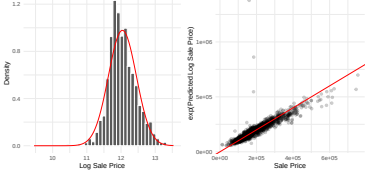
Applied Machine Learning

Feature Engineering: Target Transformations



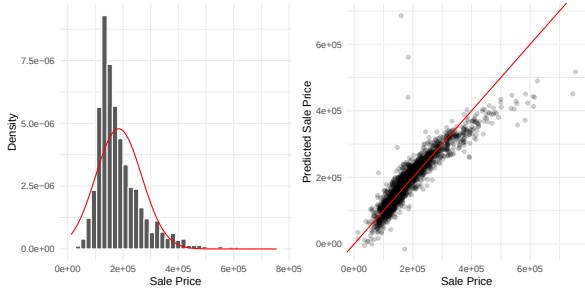
Learning goals

- Why and when to transform the target
- Common transformations: log, Box-Cox, Yeo-Johnson
- The transform-fit-invert pipeline

WHY TRANSFORM THE TARGET?

Previous slides transformed *features* (**x**); here we transform the *target* (y). Predictions must then be inverted to the original scale.

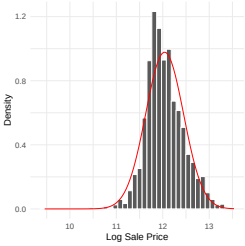
When targets are heavily skewed, regression models struggle: large tail residuals dominate squared-error loss, error variance grows with y (heteroscedasticity), and the model "chases" extremes.

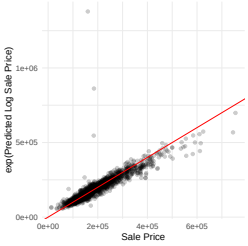


Ames housing data: sale prices are right-skewed (left). Predictions of a linear model fit poorly at the extremes (right) because tail residuals dominate the squared-error loss.

LOG TRANSFORMATION

The simplest fix: model $\log(y)$ instead of y, then exponentiate predictions back.





Why it works: the log compresses the long right tail, making the error distribution more symmetric and stabilizing its variance.

Variants:

- $\log(y)$ – requires $y > 0$
- $\log(y + 1)$ – handles zeros (common for count data)
- $\log(y + c)$ – shift by constant c for negative or zero y

THE TRANSFORM-FIT-INVERT PIPELINE

Target transformations are **not** a preprocessing step – they are part of the model:

1. **Transform** the target: $\tilde{y} = f(y)$ (e.g., $f = \log$)
2. **Fit** the model on transformed targets: $\hat{\tilde{y}} = g(\mathbf{x})$
3. **Invert** predictions back to the original scale: $\hat{y} = f^{-1}(\hat{\tilde{y}})$

Important: the transformation parameters (e.g., λ for Box-Cox) must be estimated on the **training fold only**. Fitting the transformation on the full dataset before splitting leaks information and produces overly optimistic CV results.

In scikit-learn: wrap with `TransformedTargetRegressor(regressor, func=..., inverse_func=...)`
In R/mlr3: use `PipeOpTargetTrafo` inside a pipeline (see time series chapter for a worked example)

POWER TRANSFORMS: BOX-COX AND YEO-JOHNSON

Instead of choosing log by hand, power transforms find the best transformation automatically by optimizing a parameter λ:

Box-Cox (requires $y > 0$):

$$y^{(\lambda)} = \begin{cases} \dfrac{y^{\lambda} - 1}{\lambda} & \lambda \neq 0 \\ \log(y) & \lambda = 0 \end{cases}$$

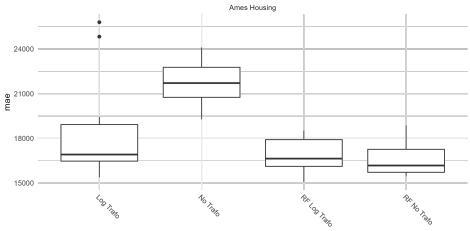
Yeo-Johnson (any y):

$$y^{(\lambda)} = \begin{cases} \dfrac{(y+1)^{\lambda} - 1}{\lambda} & \lambda \neq 0,\ y \geq 0 \\ \log(y+1) & \lambda = 0,\ y \geq 0 \\ -\dfrac{(-y+1)^{2-\lambda} - 1}{2-\lambda} & \lambda \neq 2,\ y < 0 \\ -\log(-y+1) & \lambda = 2,\ y < 0 \end{cases}$$

- λ is fit via maximum likelihood; Box-Cox special cases: $\lambda = 1$ (identity), 0 (log), 0.5 ($\sqrt{y}$), -1 ($1/y$)
- **Decision rule:** use Box-Cox when $y > 0$ is guaranteed, Yeo-Johnson otherwise

scikit-learn: `PowerTransformer(method='yeo-johnson'|'box-cox')`; R: `recipes::step_BoxCox()`, `step_YeoJohnson()`

BENCHMARK: DOES IT HELP?



Ames housing data, 10-fold CV, MAE.

Log-transforming the target substantially improves the linear model but barely changes random forest.

Practical takeaway:

- Linear and distance-based models benefit most
- Tree-based models can fit skewed targets natively (split thresholds adapt to the data); gradient boosting or random forests often do not need a target transform
- Always benchmark with vs. without to verify it helps

SUMMARY

- Skewed targets cause heteroscedastic errors that hurt models with squared-error loss
- Common target transformations: $\log(y)$, $\log(y+1)$, Box-Cox ($y > 0$), Yeo-Johnson (any y)
- The transformation is part of the model pipeline (transform $\rightarrow$ fit $\rightarrow$ invert), not a preprocessing step – parameters must be estimated per fold to avoid leakage
- Tree-based models adapt their splits to the target distribution and rarely benefit from a target transform; the benefit is greatest for linear and distance-based models
- The same principle applies to **detrending in time series**: subtract a trend (target transformation), model residuals, add the trend back at prediction time $\rightarrow$ see time series chapter